College of Basic and Applied Sciences
School of Engineering Sciences · Department of Computer Engineering

CPEN 208 — Project 1

Relational Database & Web Application
for the Computer Engineering Department

Name: David Kwame Odoi-Anim
Student ID: 22312110
Course: CPEN 208 — Introduction to Software Engineering
Semester: Second Semester, 2025/2026
Lecturer: Mr. Asiamah

PostgreSQL · Next.js 14 · TypeScript · Tailwind CSS

GitHub: `https://github.com/doxaodoi/project-1-2`

1. Overview

This project delivers a relational database for a software the Computer Engineering Department intends to build, together with a Next.js 14 web front end. The system supports five functionalities required by the brief:

#	Functionality	Implemented by
1	Student personal information	`people.students`
2	Student fees payments	`finance.fee_bills`, `finance.payments`
3	Course enrollment	`academic.enrollments`
4	Lecturers to course assignment	`academic.lecturer_course_assignment`
5	Lecturers to TA assignment	`academic.lecturer_ta_assignment`

A database function, `finance.get_outstanding_fees()`, computes each student's outstanding balance and returns it as a JSON array, as required.

1.1 Assumptions

- Database name: `coe_dept`. The design is split into four schemas for clarity: `people`, `academic`, `finance` and `auth`.
- The sample data is the real CPEN 208 class roster (70 students). Each student's email follows the University pattern `<studentID>@st.ug.edu.gh`.
- Supporting attributes not present in the roster (phone, date of birth) are generated sample values; gender is left unspecified rather than inferred from names.
- Each student is billed three fee items per semester (Tuition, Academic Facility User Fee, SRC & Other Dues) totalling **GH₵12,300.00**; payments vary so that outstanding balances are realistic.
- Every student has a login account (`auth.users`) with the demo password `Student@123` for grading.

2. Database Design

The database is organised into four schemas. The tables and their most important columns are shown below; primary keys are underlined in the SQL scripts and every relationship is enforced with a foreign key.

people

programs

program_id (PK), code, name, degree

students

student_id (PK), full_name, email, phone, date_of_birth, level, program_id → programs

lecturers

lecturer_id (PK), staff_no, title, full_name, email, academic_rank

teaching_assistants

ta_id (PK), full_name, email, student_id → students

academic

courses

course_id (PK), code, title, credit_hours, level, semester

enrollments

enrollment_id (PK), student_id → students, course_id → courses, academic_year, semester, grade

lecturer_course_assignment

lecturer_id → lecturers, course_id → courses, academic_year, semester

lecturer_ta_assignment

lecturer_id → lecturers, ta_id → teaching_assistants, course_id → courses

finance

fee_bills

bill_id (PK), student_id → students, academic_year, semester, description, amount_due

payments

payment_id (PK), student_id → students, amount, paid_on, method, reference

auth

users

user_id (PK), student_id → students (unique), email (unique), password_hash, role

2.1 Relationships (foreign keys)

Child table	Column	References
people.students	program_id	people.programs
people.teaching_assistants	student_id	people.students
academic.enrollments	student_id, course_id	people.students, academic.courses

academic.lecturer_course_assignment	lecturer_id, course_id	people.lecturers, academic.courses
academic.lecturer_ta_assignment	lecturer_id, ta_id, course_id	people.lecturers, people.teaching_assistants, academic.courses
finance.fee_bills	student_id	people.students
finance.payments	student_id	people.students
auth.users	student_id	people.students

2.2 Build scripts

The database is created by six ordered scripts in `PROJECT 1/database`, run in the pgAdmin Query Tool:

Script	Purpose
00_create_database.sql	Creates the `coe_dept` database
01_schemas.sql	Creates the four schemas
02_tables.sql	Creates all tables, keys and indexes
03_functions.sql	Creates the outstanding-fees functions
04_seed.sql	Inserts the 70-student class roster and all sample data
05_verify.sql	Row-count checks and a demonstration of the JSON function

3. The Outstanding-Fees Function

The required function aggregates each student's total bills and total payments and returns `billed − paid` as the outstanding amount, for every student, in a single JSON array.

```
CREATE OR REPLACE FUNCTION finance.get_outstanding_fees()
RETURNS JSON
LANGUAGE sql
STABLE
AS $$
    SELECT COALESCE(json_agg(row_to_json(t) ORDER BY t.full_name), '[]'::json)
    FROM (
        SELECT
            s.student_id, s.full_name, s.email,
            COALESCE(b.total_billed, 0)                          AS total_billed,
            COALESCE(p.total_paid, 0)                            AS total_paid,
            COALESCE(b.total_billed, 0) - COALESCE(p.total_paid, 0) AS outstanding
        FROM people.students s
        LEFT JOIN (SELECT student_id, SUM(amount_due) AS total_billed
                   FROM finance.fee_bills GROUP BY student_id) b ON b.student_id =
s.student_id
        LEFT JOIN (SELECT student_id, SUM(amount) AS total_paid
                   FROM finance.payments GROUP BY student_id) p ON p.student_id =
s.student_id
    ) t;
$$;
```

3.1 Sample output

Calling `SELECT finance.get_outstanding_fees();` returns a 70-element array. Three representative elements (a full payer, a partial payer, and the author's account):

```
[
  { "student_id": 22384451, "full_name": "Abu Neaquittae Golda",
    "total_billed": 12300.00, "total_paid": 12300.00, "outstanding": 0.00 },
  { "student_id": 22357814, "full_name": "Adzasa Stephen Yaw",
    "total_billed": 12300.00, "total_paid": 8000.00,  "outstanding": 4300.00 },
  { "student_id": 22312110, "full_name": "David Kwame Odoi-Anim",
    "total_billed": 12300.00, "total_paid": 9000.00,  "outstanding": 3300.00 }
]
```

3.2 Distribution across the class

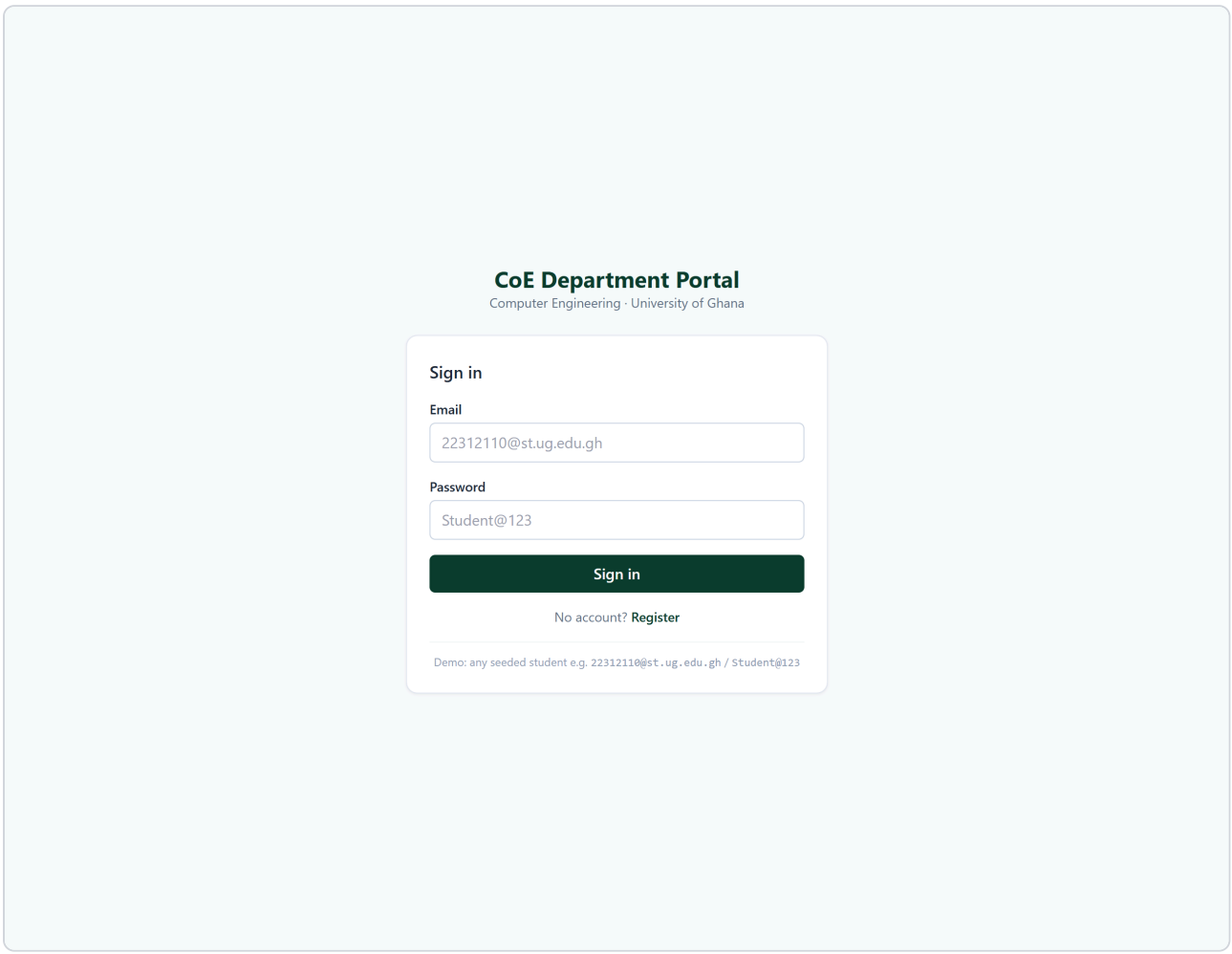
Outstanding (GH₵)	Number of students
0.00 (fully paid)	27

3,300.00	1
4,300.00	14
7,300.00	14
12,300.00 (no payment yet)	14

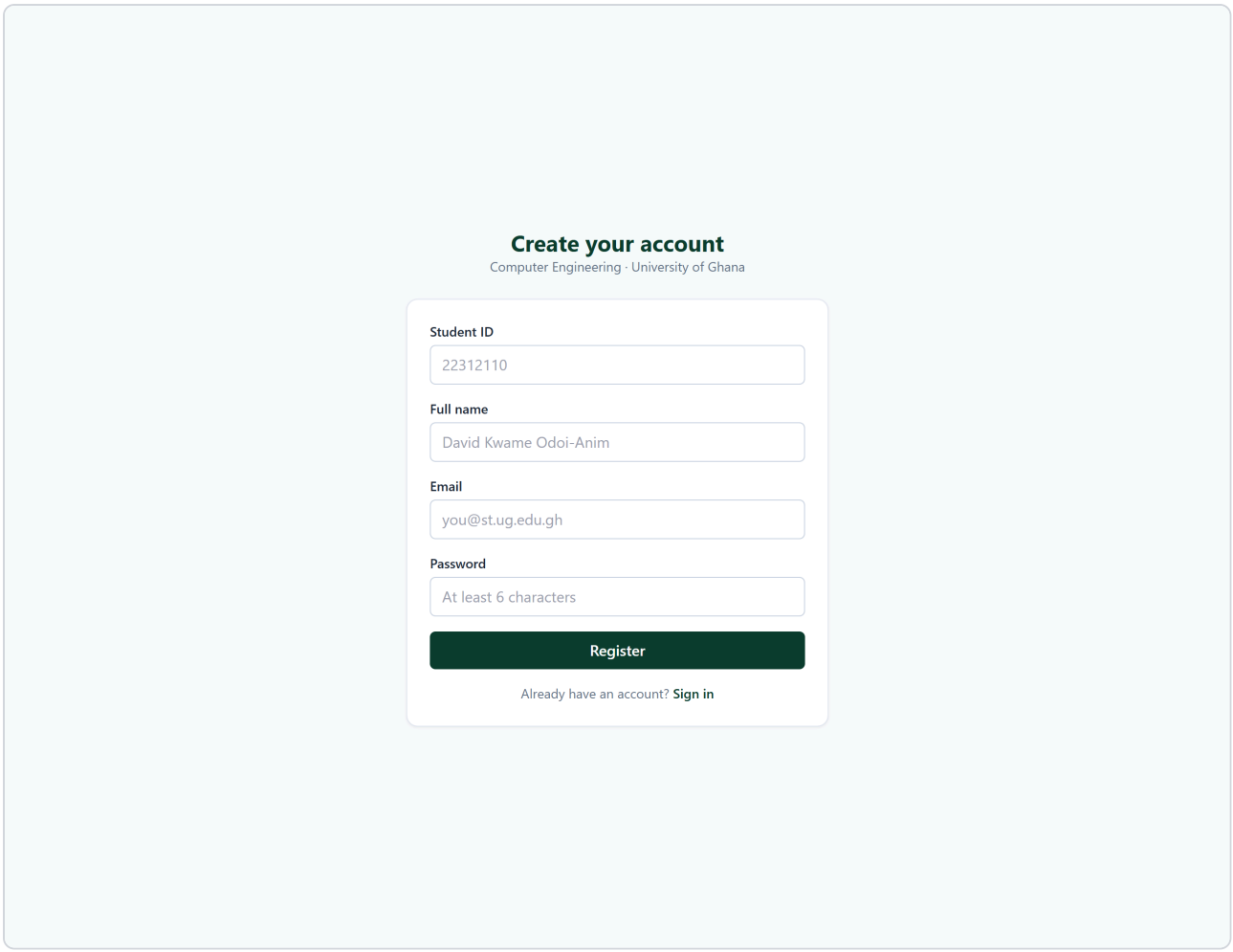
A scalar helper, `finance.get_student_outstanding(student_id)`, returns the balance for one student and is reused by the web application's dashboard.

4. Web Application (Next.js 14)

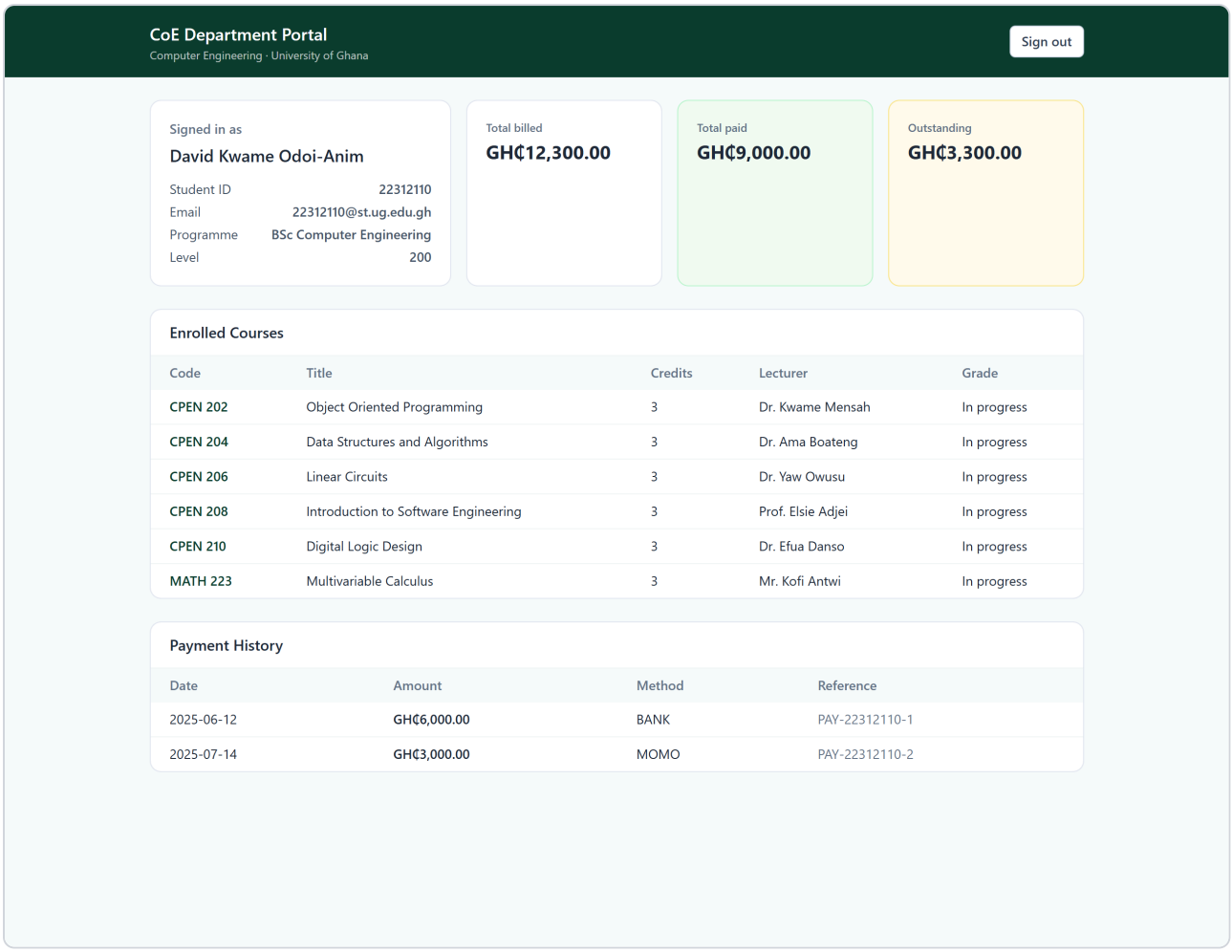
The front end is a Next.js 14 application (App Router, TypeScript, Tailwind CSS) with three pages — **login**, **register** and **dashboard**. It does not query the database directly; it consumes the Project 2 API. Authentication uses a JWT that the Next.js server stores in an HttpOnly cookie, and the `/dashboard` route is guarded by middleware.



Login page — seeded students sign in with their student-ID email and the demo password.



Registration page — creates a new student and login account.



Dashboard for student 22312110 — profile, live fee summary (billed / paid / outstanding from the database function), enrolled courses with assigned lecturers, and payment history.

5. How to Run

1. **Database:** in pgAdmin, run `00_create_database.sql` against the `postgres` database, then run `01` – `05` against `coe_dept`. (Or restore `coe_dept_backup.backup`.)
2. **API (Project 2):** `cd "PROJECT 2/api"`, set `DB_PASSWORD`, then `mvn spring-boot:run` (starts on port 8080).
3. **Front end:** `cd "PROJECT 1/web"`, copy `.env.example` to `.env.local`, then `npm install && npm run dev` and open `http://localhost:3000`.
4. Sign in with `22312110@st.ug.edu.gh` / `Student@123`.

Deliverables in this folder: the SQL scripts and database backup are in `PROJECT 1/database`; the front-end source is in `PROJECT 1/web`; all source code is on GitHub at `https://github.com/doxaodoi/project-1-2`.